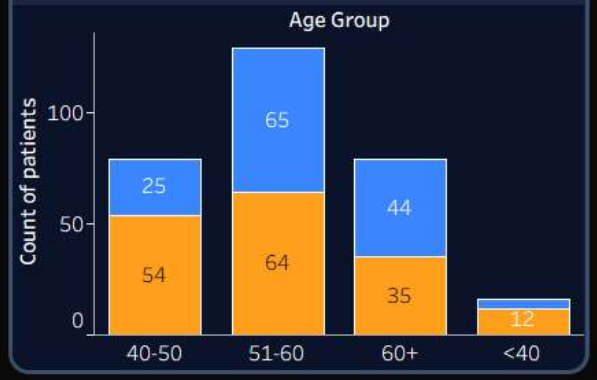
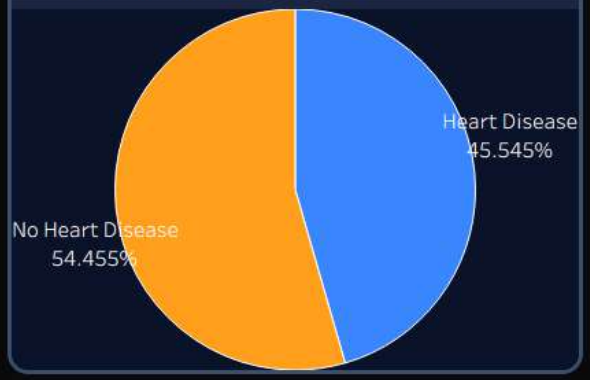
Healthcare(Heart Diesease) Analysis

Heart Disease Label
Heart Disea.. No Heart Di..

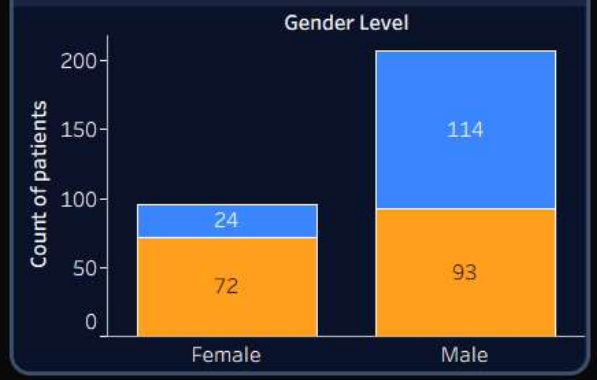
Heart Disease by Age Group
Which age group is most affected



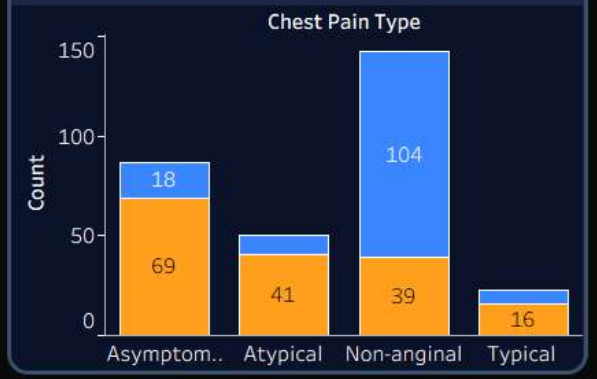
Heart Disease Distribution
% of patients with vs without heart disease



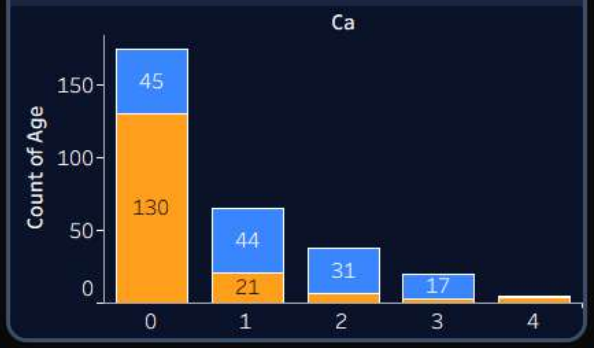
Heart Disease by Gender
Gender-wise risk comparison



Chest Pain Type vs Heart Discase
Non-anginal patients often show higher risk



Major Vessels (ca) vs Heart Disease
Higher number of vessels -+ higher disease probability



Exercise Induced Angina Impact
Exercise-induced angina is a strong indicator

Age Group
(All)

Chest Pain Type
(All)

Exercice Practice
(All)

Gender Level
(All)

Heart Disease Label
(All)

1. Heart disease prevalence increases significantly after age 50.
2. Males show a higher incidence of heart disease compared to females.
3. Patients with **asymptomatic chest pain** have higher heart disease risk
4. Lower maximum heart rate is strongly associated with heart disease.
5. Exercise-induced angina is a major risk indicator.
6. Patients with more major vessels colored (ca 2 1) show higher disease probability.
7. Cholesterol alone i..

🔥 Healthcare Analytics: Predicting Cardiovascular Disease Risk

🎯 OBJECTIVE

`Analyze patient data to identify trends` `Improve hospital decision-making` `Track patient demographics, diseases, and treatment outcomes` `Build interactive dashboards for stakeholders`

🧰 TOOLS USED

`Python (Pandas, Matplotlib, Seaborn), Jupyter Notebook, Tableau`

1. Structure of data, Checking the missing values, duplicate records

🧠 STEP 1: IMPORT LIBRARIES

In [3]:
```
# Step 1: Importing necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
print('imported')
```

```
imported
```

📂 STEP 2: LOAD DATA

In [4]:
```
# loding the dataset
data=pd.read_excel('data1.xlsx')
# Checking the first few rows
data.head()
```

Out[4]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

🔍 STEP 3: DATA UNDERSTANDING

In [5]:

```
# Check dataset info
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    int64
 1   sex       303 non-null    int64
 2   cp        303 non-null    int64
 3   trestbps  303 non-null    int64
 4   chol      303 non-null    int64
 5   fbs       303 non-null    int64
 6   restecg   303 non-null    int64
 7   thalach   303 non-null    int64
 8   exang     303 non-null    int64
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    int64
 11  ca        303 non-null    int64
 12  thal      303 non-null    int64
 13  target    303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

In [6]:
```
# Check null values
data.isnull().sum()
```

Out[6]:
```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

In [7]:
```
# Describe data
data.describe()
```

Out[7]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.00000
mean	54.366337	0.683168	0.966997	131.623762	246.264026	0.148515	0.528053	149.646865	0.326733	1.039604	1.39934
std	9.082101	0.466011	1.032052	17.538143	51.830751	0.356198	0.525860	22.905161	0.469794	1.161075	0.61622
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000	71.000000	0.000000	0.000000	0.00000
25%	47.500000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000	133.500000	0.000000	0.000000	1.00000
50%	55.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.000000	153.000000	0.000000	0.800000	1.00000
75%	61.000000	1.000000	2.000000	140.000000	274.500000	0.000000	1.000000	166.000000	1.000000	1.600000	2.00000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000	202.000000	1.000000	6.200000	2.00000

🧹 STEP 4: DATA CLEANING

In [67]:
```
# check for duplicate
data[data.duplicated()]
```

Out[67]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
164	38	1	2	138	175	0	1	173	0	0.0	2	4	2	1

In [68]:
```
data.shape
```

Out[68]:
```
(303, 14)
```

In [70]:
```
# Remove duplicates
data.drop_duplicates(inplace=True)
data.shape
```

Out[70]:
```
(302, 14)
```

EDA -Univariate data

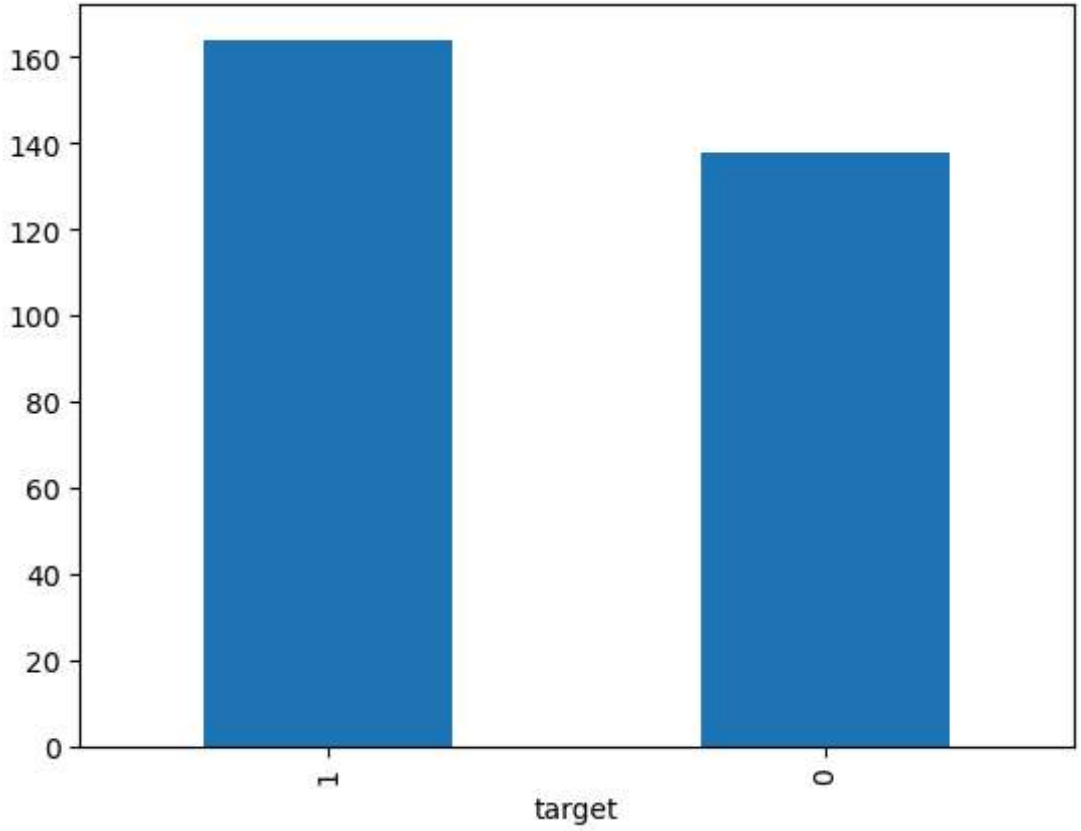
📈 STEP 5: DATA ANALYSIS

In [15]:
```
# target col
data['target'].value_counts()
```

Out[15]:
```
target
1    164
0    138
Name: count, dtype: int64
```

📊 STEP 6: DATA VISUALIZATION

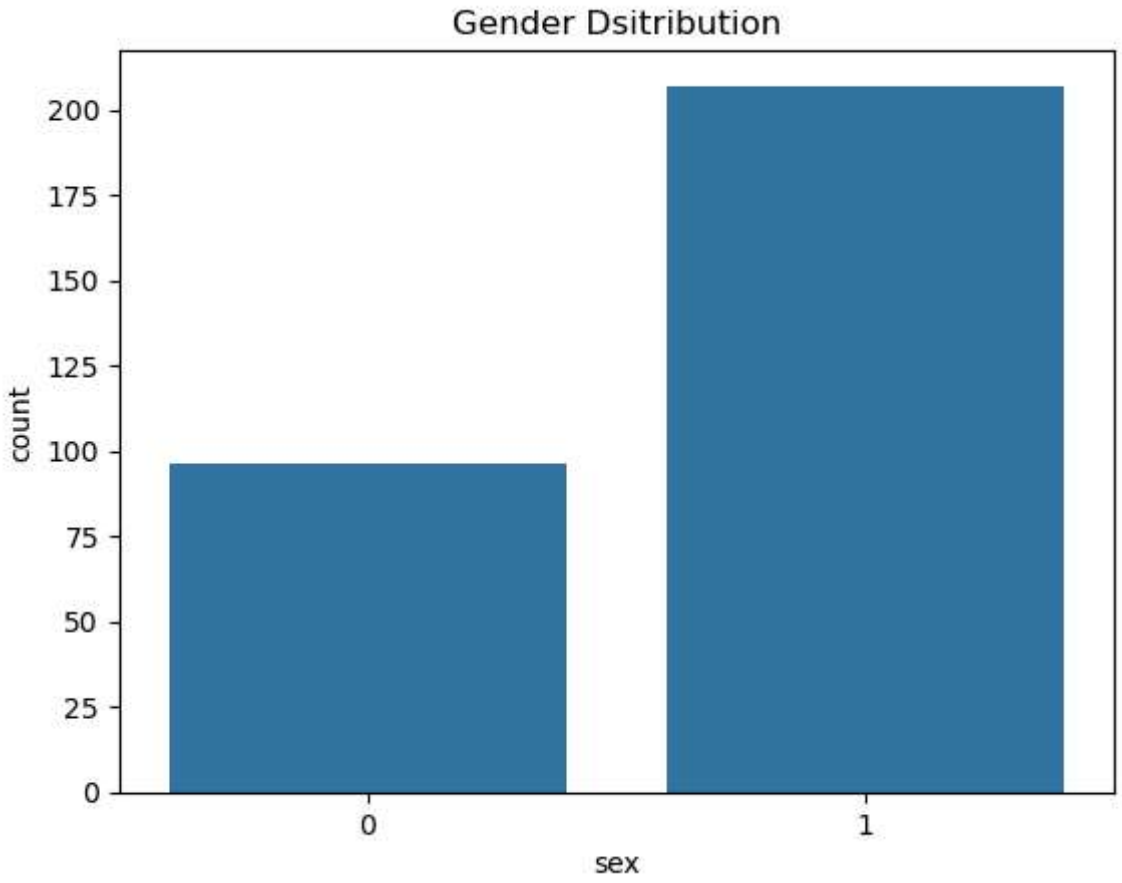
In [19]:
```
data['target'].value_counts().plot(kind='bar')
plt.show()
```



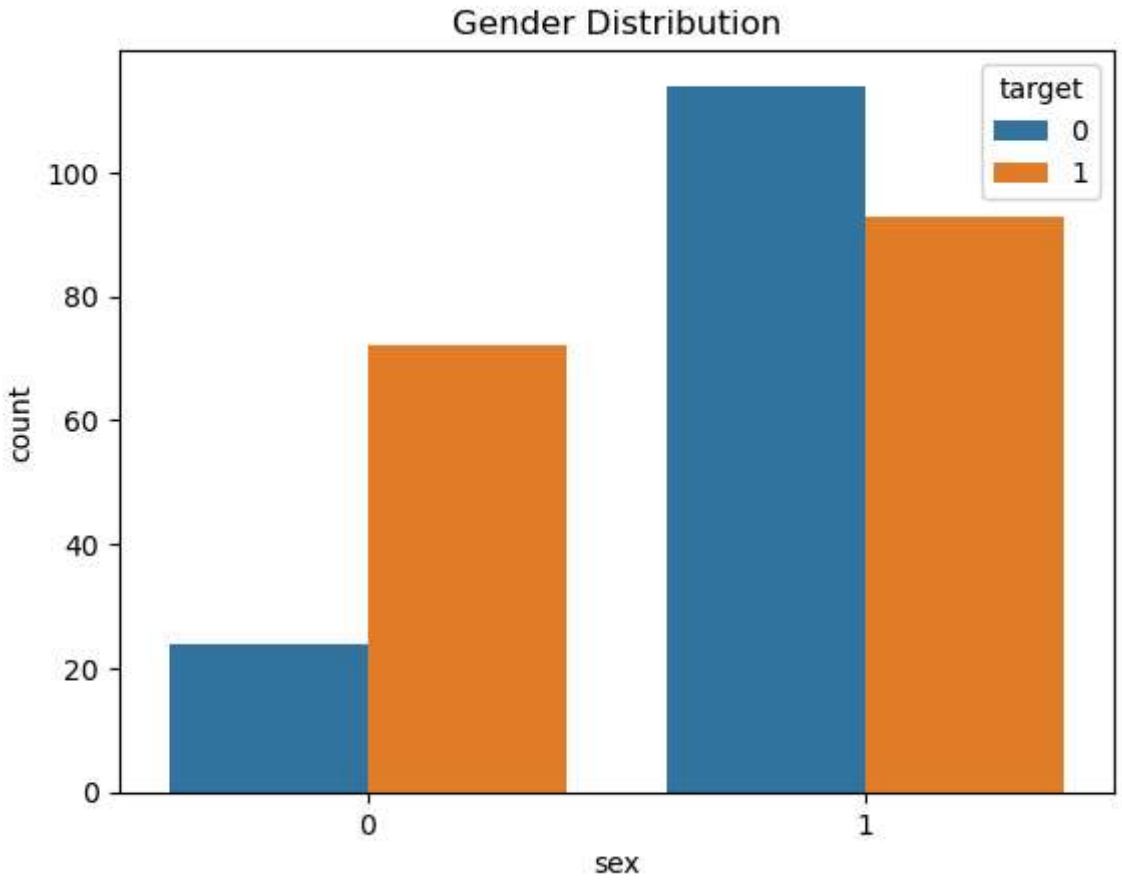
In [8]:
```
data['sex'].value_counts()
```

Out[8]:
```
sex
1    207
0     96
Name: count, dtype: int64
```

In [9]:
```
# gender col
sns.countplot(x=data['sex'])
plt.title('Gender Dsitribution')
plt.show()
```



In [10]:
```
sns.countplot(x=data['sex'],hue='target',data=data)
plt.title('Gender Distribution')
plt.show()
```

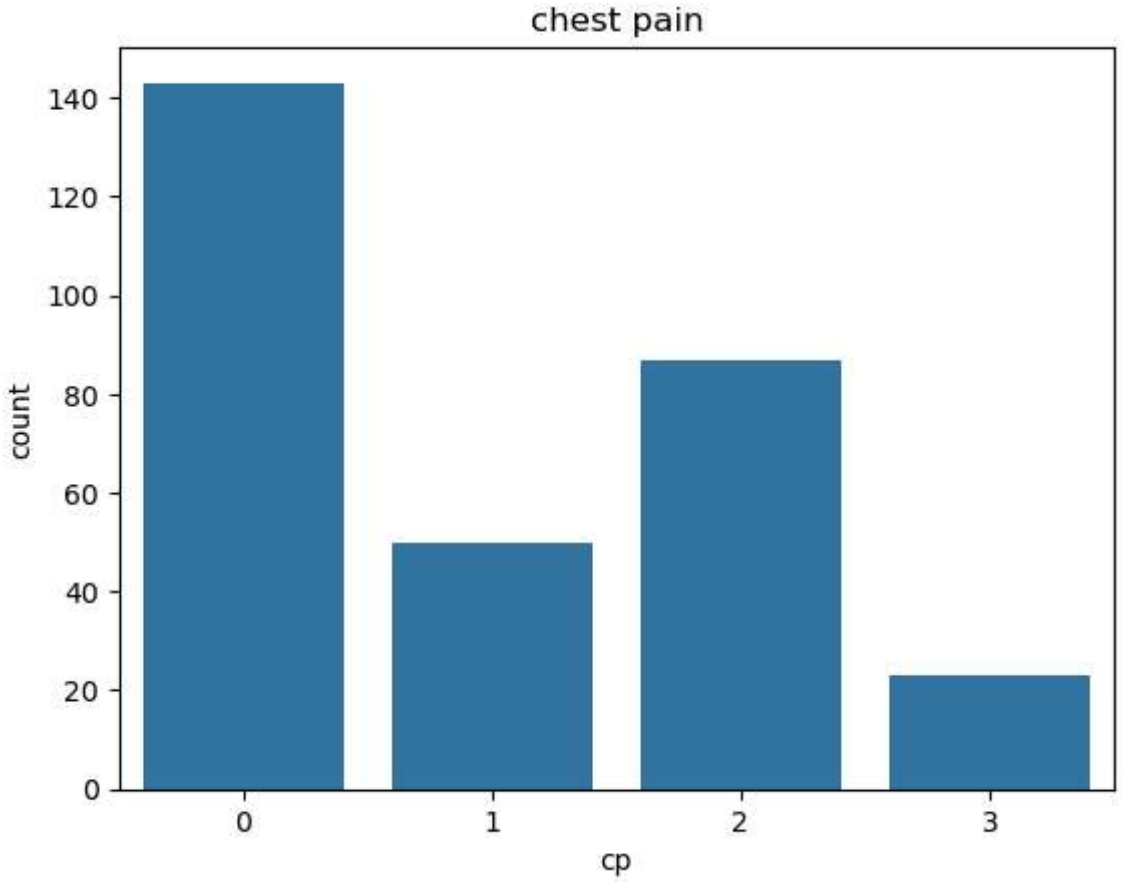


chest pain type (0 = Non anginal; 1 = Non typical; 2 = Asymptomatic; 3 = Typical)

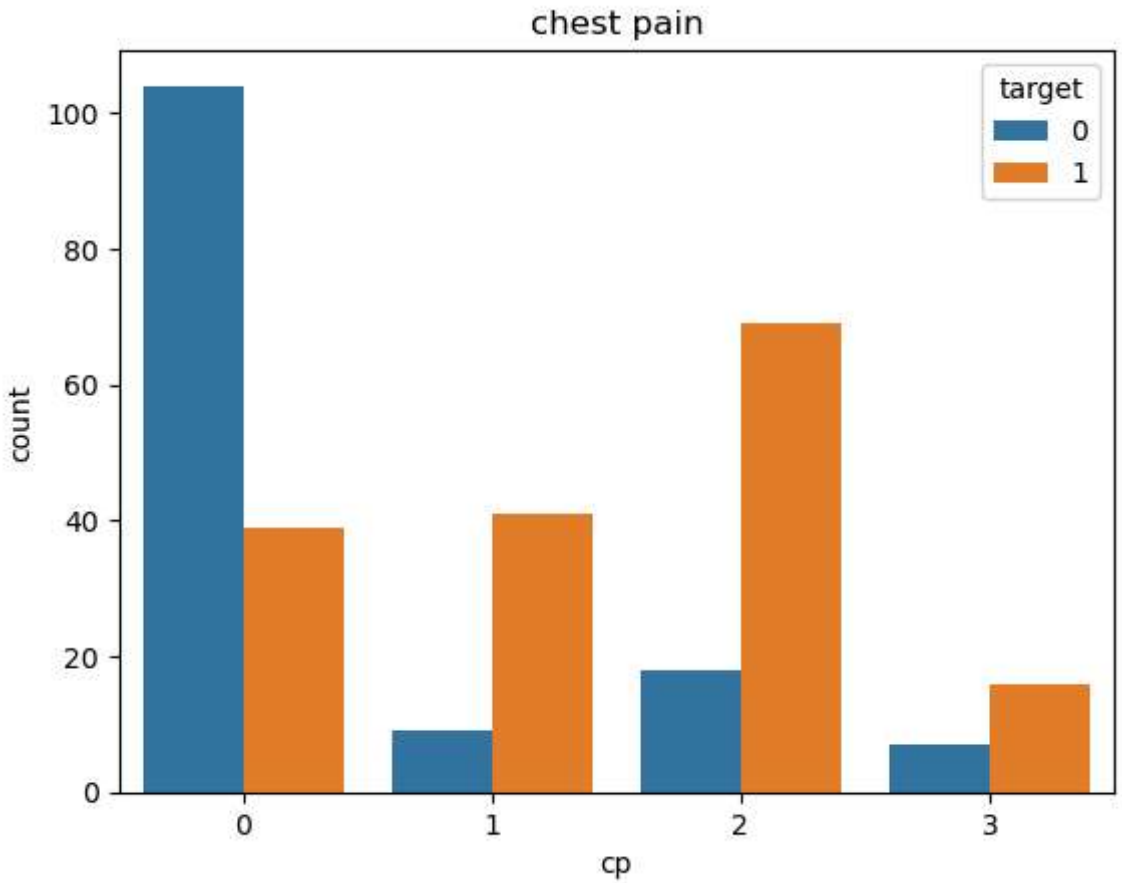
In [11]:
```
data['cp'].value_counts()
```

Out[11]:
```
cp
0    143
2     87
1     50
3     23
Name: count, dtype: int64
```

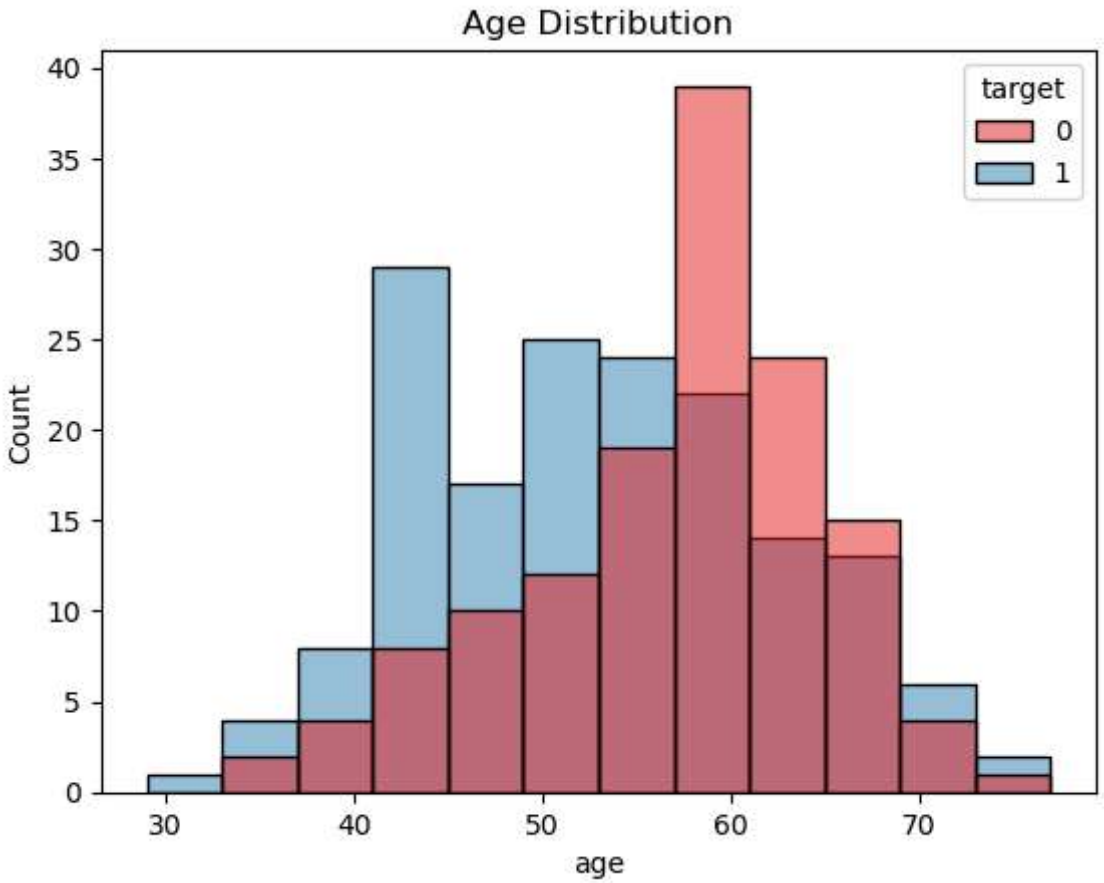
In [12]:
```
sns.countplot(x=data['cp'])
plt.title('chest pain')
plt.show()
```



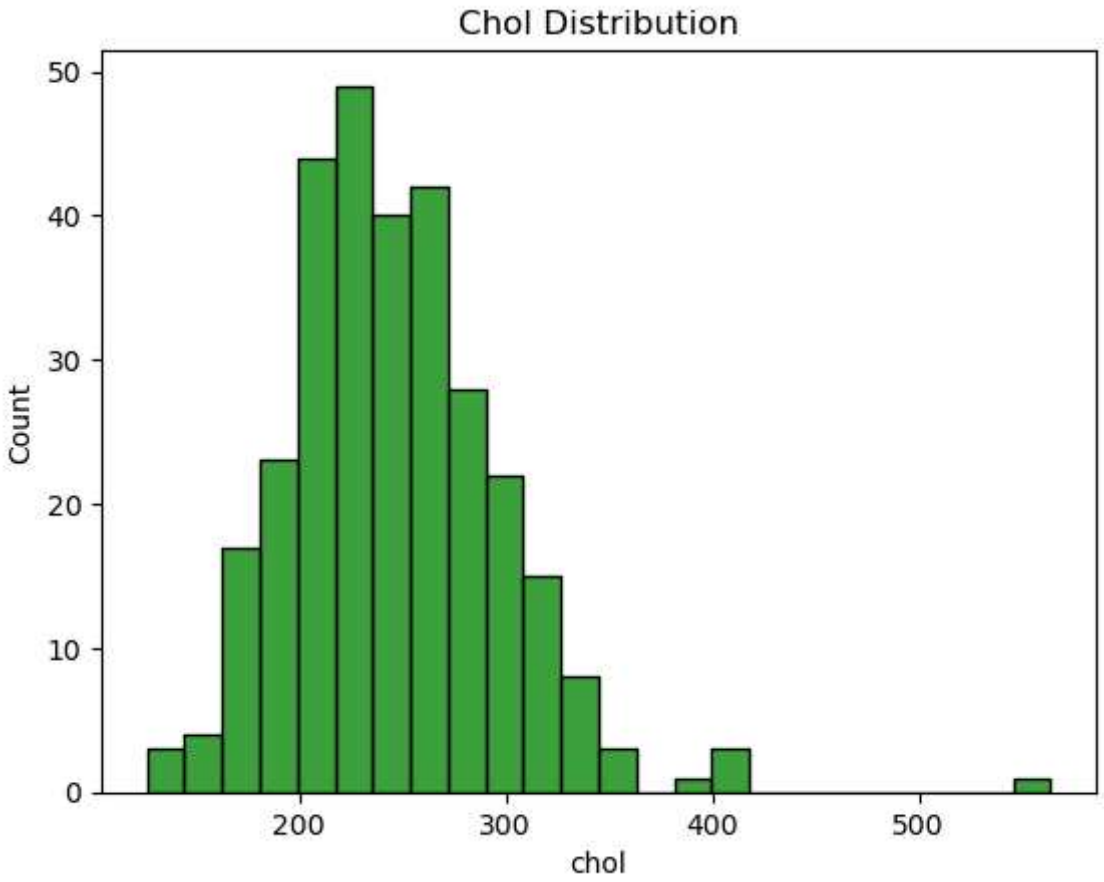
In [13]:
```
sns.countplot(x=data['cp'],hue='target',data=data)
plt.title('chest pain')
plt.show()
```



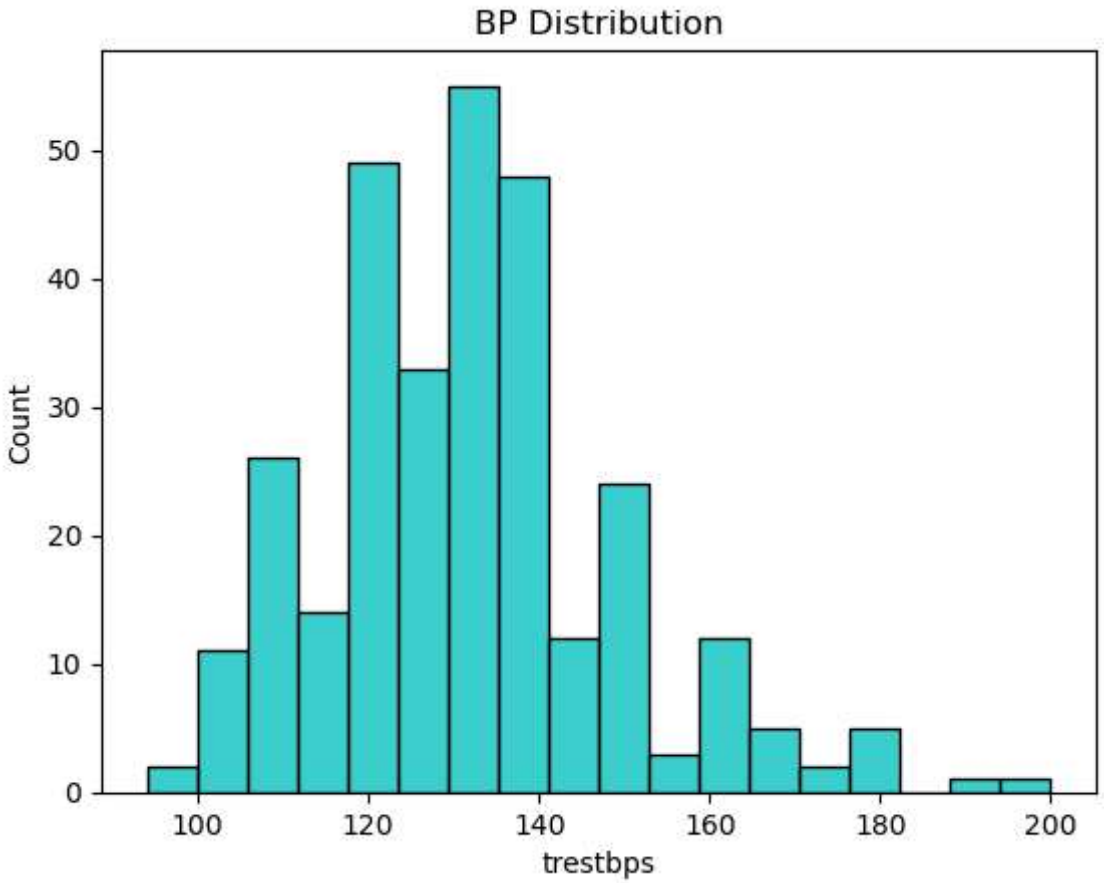
In [14]:
```
# age distribution
sns.histplot(x=data['age'],hue='target',palette='Set1', data=data)
plt.title('Age Distribution')
plt.show()
```



In [15]:
```
# chol distribution
sns.histplot(x=data['chol'],color='g')
plt.title('Chol Distribution')
plt.show()
```

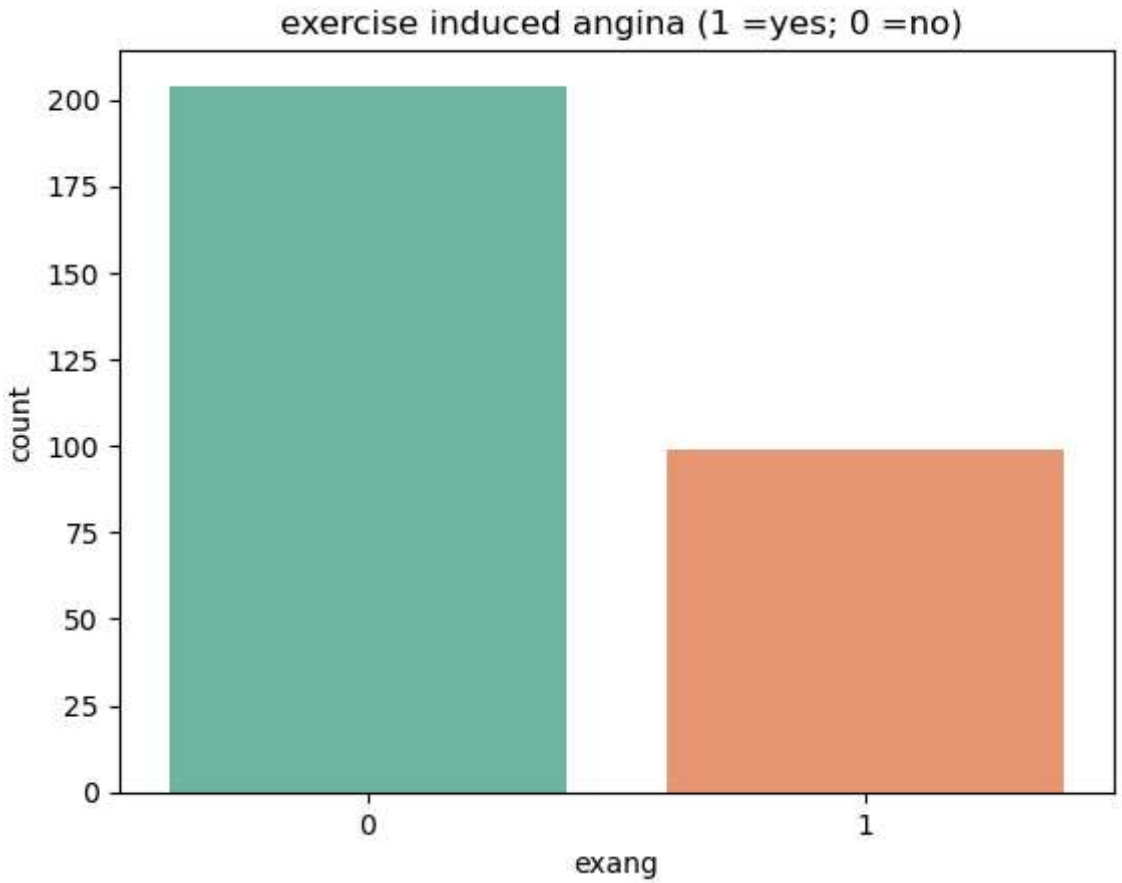


In [16]:
```
# BP distribution
sns.histplot(x=data['trestbps'],color='c')
plt.title('BP Distribution')
plt.show()
```



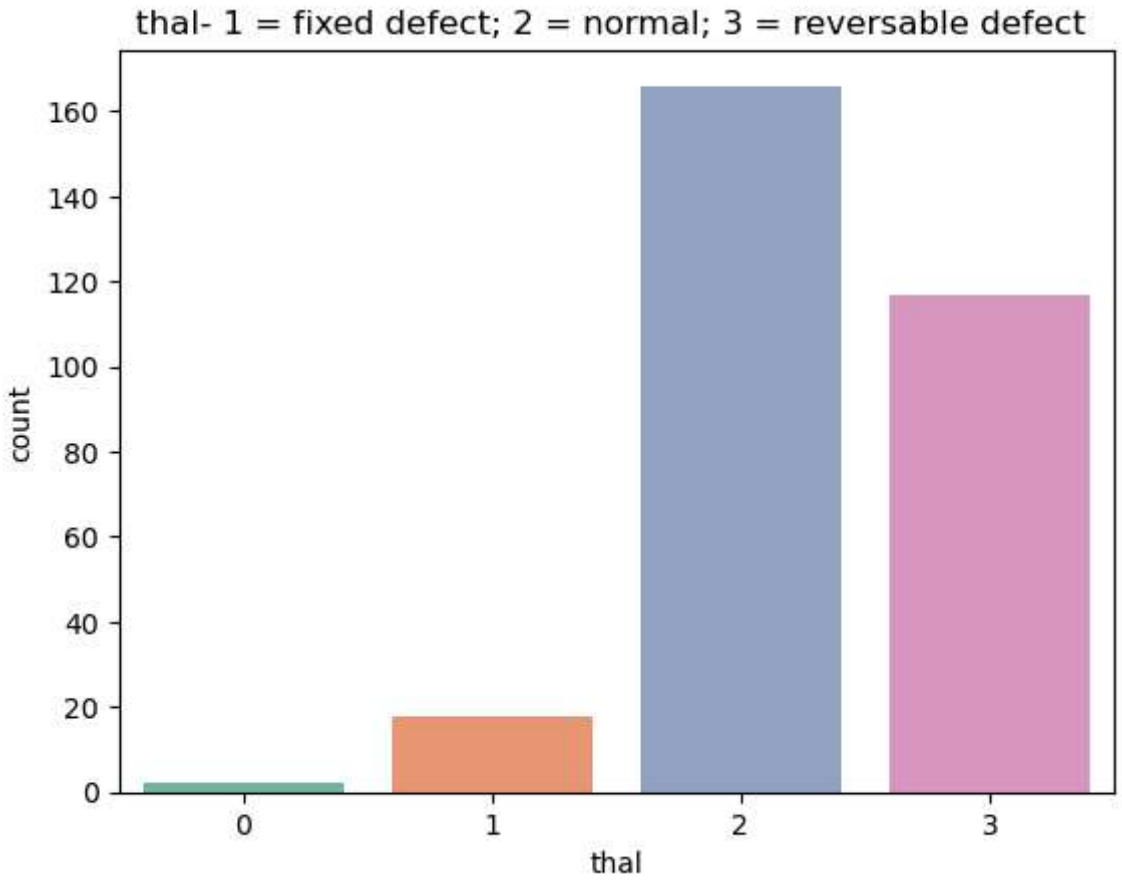
In [17]:
```
import warnings
warnings.filterwarnings('ignore')
```

In [18]:
```
#exang
sns.countplot(x=data['exang'],palette='Set2')
plt.title(' exercise induced angina (1 =yes; 0 =no)')
plt.show()
```

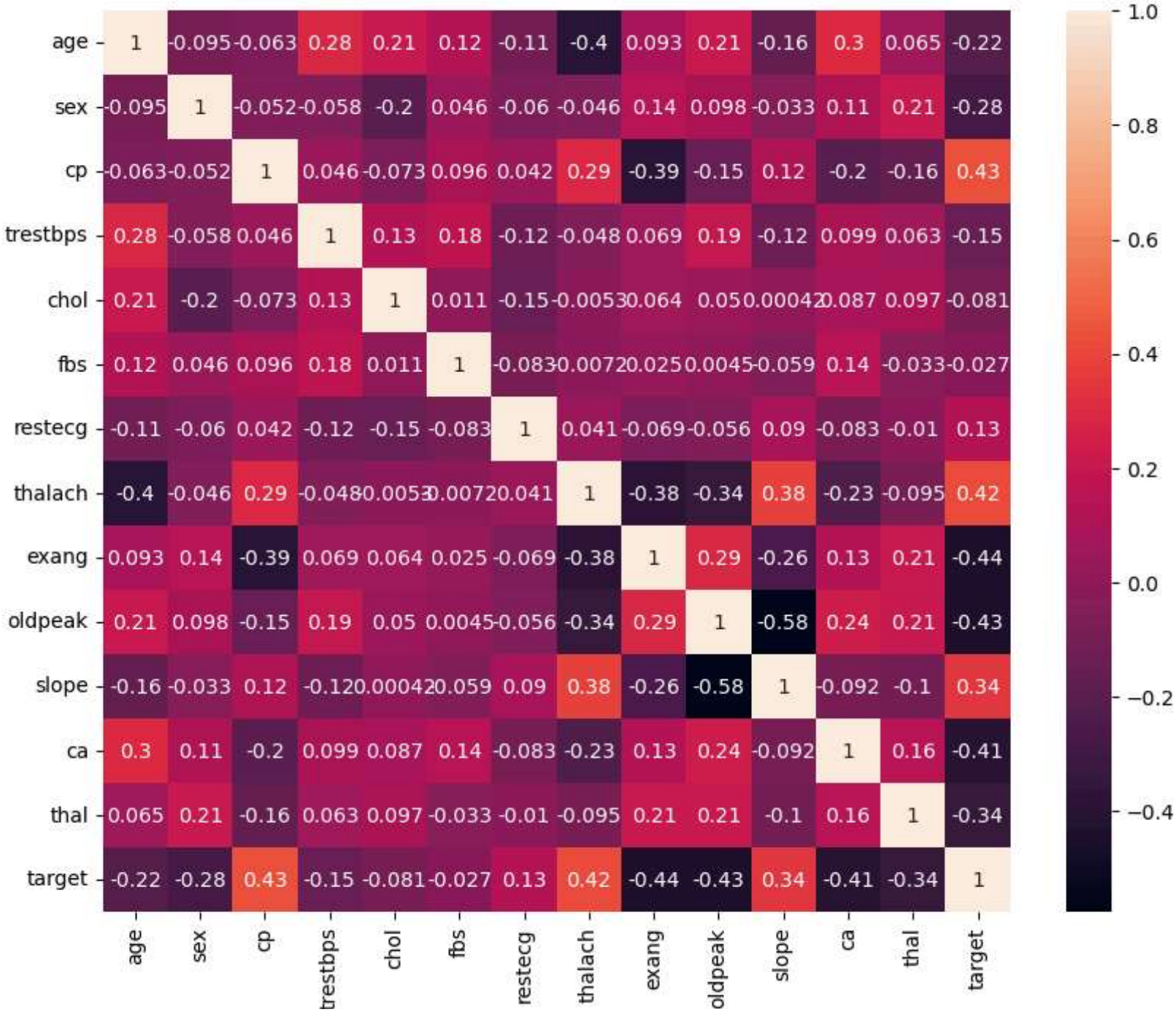


1 = fixed defect; 2 = normal; 3 = reversable defect

In [19]:
```
sns.countplot(x=data['thal'],palette='Set2')
plt.title('thal- 1 = fixed defect; 2 = normal; 3 = reversable defect ')
plt.show()
```

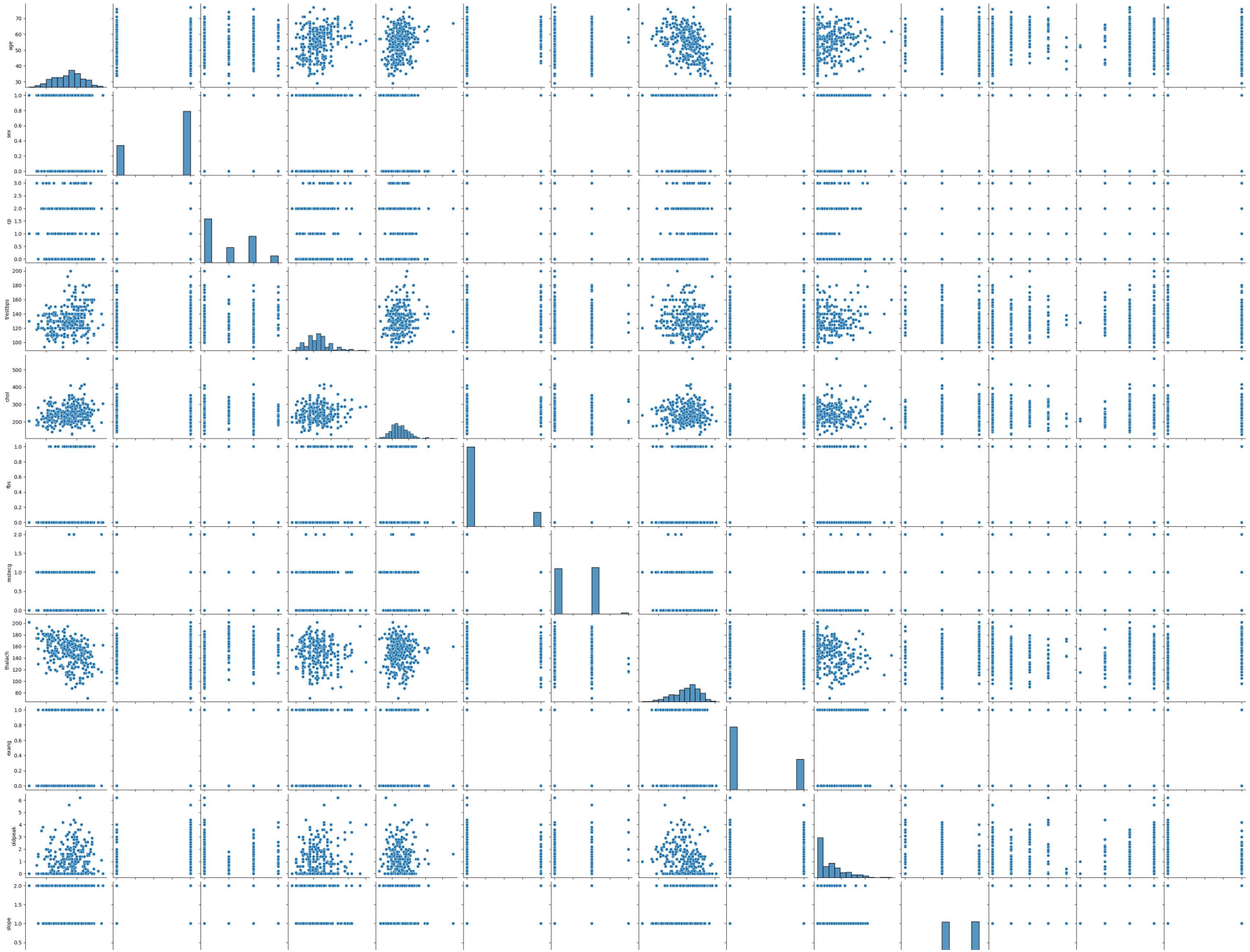


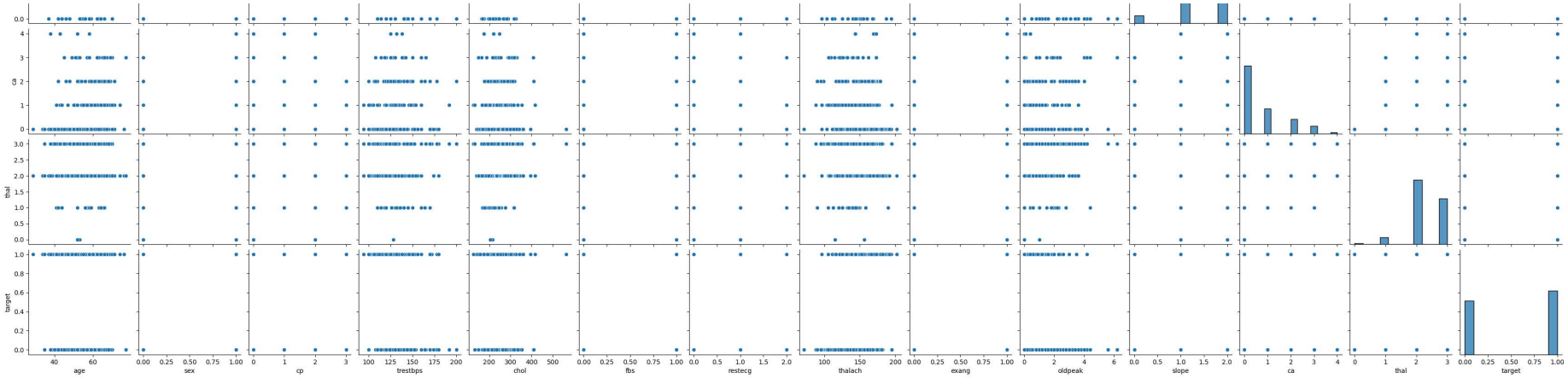
In [41]:
```
# heatmap
plt.figure(figsize=(10,8))
sns.heatmap(data.corr(),annot=True)
plt.show()
```


age 1 -0.095 -0.063 0.28 0.21 0.12 -0.11 -0.4 0.093 0.21 -0.16 0.3 0.065 -0.22
sex -0.095 1 -0.052 -0.058 -0.2 0.046 -0.06 -0.046 0.14 0.098 -0.033 0.11 0.21 -0.28
cp -0.063 -0.052 1 0.046 -0.073 0.096 0.042 0.29 -0.39 -0.15 0.12 -0.2 -0.16 0.43
trestbps 0.28 -0.058 0.046 1 0.13 0.18 -0.12 -0.048 0.069 0.19 -0.12 0.099 0.063 -0.15
chol 0.21 -0.2 -0.073 0.13 1 0.011 -0.15 -0.0053 0.064 0.05 0.00042 0.087 0.097 -0.081
fbs 0.12 0.046 0.096 0.18 0.011 1 -0.083 0.0072 0.025 0.0045 -0.059 0.14 -0.033 -0.027
restecg -0.11 -0.06 0.042 -0.12 -0.15 -0.083 1 0.041 -0.069 -0.056 0.09 -0.083 -0.01 0.13
thalach -0.4 -0.046 0.29 -0.048 0.0053 0.0072 0.041 1 -0.38 -0.34 0.38 -0.23 -0.095 0.42
exang 0.093 0.14 -0.39 0.069 0.064 0.025 -0.069 -0.38 1 0.29 -0.26 0.13 0.21 -0.44
oldpeak 0.21 0.098 -0.15 0.19 0.05 0.0045 -0.056 -0.34 0.29 1 -0.58 0.24 0.21 -0.43
slope -0.16 -0.033 0.12 -0.12 0.00042 0.059 0.09 0.38 -0.26 -0.58 1 -0.092 -0.1 0.34
ca 0.3 0.11 -0.2 0.099 0.087 0.14 -0.083 -0.23 0.13 0.24 -0.092 1 0.16 -0.41
thal 0.065 0.21 -0.16 0.063 0.097 -0.033 -0.01 -0.095 0.21 0.21 -0.1 0.16 1 -0.34
target -0.22 -0.28 0.43 -0.15 -0.081 -0.027 0.13 0.42 -0.44 -0.43 0.34 -0.41 -0.34 1
age sex cp trestbps chol fbs restecg thalach exang oldpeak slope ca thal target
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

In [45]:

```
sns.pairplot(data)
plt.show()
```


age
sex
cp
trestbps
chol
fbs
restecg
thalach
exang
oldpeak
slope



Machine Learning

In [20]:
```
# create indep & dep vars
X=data.iloc[:, :- 1]
y=data.iloc[:,-1]
```

STEP 7: Create train data & test data

In [21]:
```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size =.20,random_state=42)
```

In [22]:
```
print('train data shape',X_train.shape)
print('test data shape',X_test.shape)
```

```
train data shape (242, 13)
test data shape (61, 13)
```

In [23]:
```
# Create Logistic Regression model
from sklearn.linear_model import LogisticRegression
log_reg=LogisticRegression()
```

In [24]:
```
# training
log_reg.fit(X_train,y_train)
```

```
y_pred=log_reg.predict(X_test)
print('Training Complted' )
```

```
Training Complted
```

In [25]:
```
y_pred
```

Out[25]:
```
array([0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0,
       0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0], dtype=int64)
```

In [27]:
```
from sklearn.metrics import confusion_matrix, accuracy_score
print(confusion_matrix(y_test,y_pred))
```

```
[[25  4]
 [ 3 29]]
```

In [28]:
```
# Accuracy of model
print(accuracy_score(y_test,y_pred))
```

```
0.8852459016393442
```

💾 STEP 8: SAVE CLEANED DATA

In [29]:
```
# Save cleaned dataset
data.to_csv("cleaned_data.csv", index=False)
```